

PROJECT DOCUMENTATION

AuraIntel: Multilingual Call Intelligence Platform

1. Project Overview

AuraIntel is a full-stack Next.js web application engineered to process call recordings (customer support and sales audio) and automatically generate deep intelligence. The platform transcribes multilingual calls, differentiates speakers (Agent vs. Customer), highlights transcripts dynamically as audio plays, and extracts executive summaries, action items, sentiment indices, and outcome classifications.

2. Core Technologies & Architecture

Framework: Next.js 15 (App Router). Used for serving frontend pages and housing serverless backend API endpoints.

Language: TypeScript. Enforces compile-time type safety across database entities, service layers, and React state.

Speech-to-Text: AssemblyAI SDK. Primary engine offering automated language detection, speaker diarization, and millisecond timestamps.

AI Insights: Google Gemini 2.5 Flash API (@google/genai). Extracted summaries, action tasks, and metadata using structured JSON schemas.

Database: Lightweight File-system DB (data/db.json). Standard JSON reads and writes, facilitating zero-configuration portability.

UI/Styling: Vanilla CSS with CSS Variables. Custom styles representing dark mode, glassmorphism, responsive grids, and highlight animations.

3. Step-by-Step Implementation Timeline

1. Research: Analyzed Windows environment paths, validated Git execution path (C:\Program Files\Git\cmd\git.exe), and tested Google Gen AI library exports.
2. Scaffolding: Initialized Next.js 15 project in scratch directory with App Router and TypeScript, bypassing Tailwind in favor of custom CSS.
3. Dependency Setup: Installed NPM packages: @google/genai, assemblyai, lucide-react, and pdfkit.
4. Data Layer: Implemented a robust data utility (src/lib/db.ts) defining standard data interfaces (CallRecord, CallInsights, ActionItem) and CRUD file operations.
5. Transcriber Adapter: Coded src/lib/audio.ts supporting AssemblyAI and a robust fallback to Gemini Files API (handling uploads, polling for ACTIVE, and parsing JSON transcript structures).
6. Insights Adapter: Coded src/lib/insights.ts sending transcripts to Gemini 2.5 Flash to automatically detect speakers, map Agent/Customer roles, and extract summaries and action items.
7. Process Orchestrator: Linked transcription and insights in src/lib/process.ts to process audio records in the background.
8. API Routes: Built REST routes: GET/POST uploads, GET calls list, GET/DELETE call details, and POST process retries.
9. Styles & Layouts: Wrote globals.css utilizing CSS variables for smooth background gradients, scrollbars, glowing metrics, and layouts.
10. Frontend Dashboard: Wrote page.tsx featuring state variables for search filters, file uploads, audio slider playback, and synced karaoke highlights.
11. GitHub setup: Created scripts/git-push.js to automate repository creation via GitHub API and push files using local Git.

Compilation Check: Built the production code successfully via npm run build, validating typescript types and route rendering.

13. Git Publishing: Executed push script using the user token, successfully creating mohdafsar313/multilingual-call-intel, followed by a secure Git origin URL reset.

4. Executed Shell Commands

Check Tools

```
node -v; npm -v; git --version
```

Project Scaffold

```
npx -y create-next-app@latest multilingual-call-intel --ts --no-tailwind --eslint --app --src-dir --import-alias "@/*" --use-npm --disable-git
```

Install Packages

```
npm install @google/genai assemblyai lucide-react pdfkit
```

Clean Test Scripts

```
Remove-Item scripts/test-sdk.js
```

Production Build Check

```
npm run build
```

GitHub Push execution

```
$env:GITHUB_TOKEN="ghp_***"; $env:GITHUB_REPO="multilingual-call-intel"; node scripts/git-push.js
```

Reset Remote for Security

```
& "C:\Program Files\Git\cmd\git.exe" remote set-url origin https://github.com/mohdafsar313/multilingual-call-intel.git
```

5. Project Core Source Code File Directory

src/lib/db.ts

Main interfaces (CallRecord, CallInsights, ActionItem, TranscriptTurn) and synchronous JSON write/read database logic.

src/lib/audio.ts

Speech-to-Text orchestrator. Integrates AssemblyAI for transcription/diarization and falls back to Gemini Files API.

src/lib/insights.ts

Generates summary, action items, customer intent, sentiment and maps speaker IDs to Agent/Customer roles using Gemini.

src/lib/process.ts

Processes audio asynchronously by chaining audio.ts (STT) and insights.ts (AI) and saving updates to the database.

src/app/page.tsx

React Client Component for dashboard UI. Manages lists, search filtering, audio tracking, and tab layouts.

src/app/globals.css

Custom style system, responsive grid layouts, scrollbars, glowing borders, and animations.

scripts/git-push.js

Automated script to create public GitHub repos via REST and run Git checkout, remote configure, and push.